

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
```

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-dataset/11490434/11490434>  0s 0us/step

```
print("Training data shape:", x_train.shape)
print("Test data shape:", x_test.shape)
```

Training data shape: (60000, 28, 28)
Test data shape: (10000, 28, 28)

```
x_train = x_train.reshape((x_train.shape[0], 28, 28, 1))
x_test = x_test.reshape((x_test.shape[0], 28, 28, 1))
```

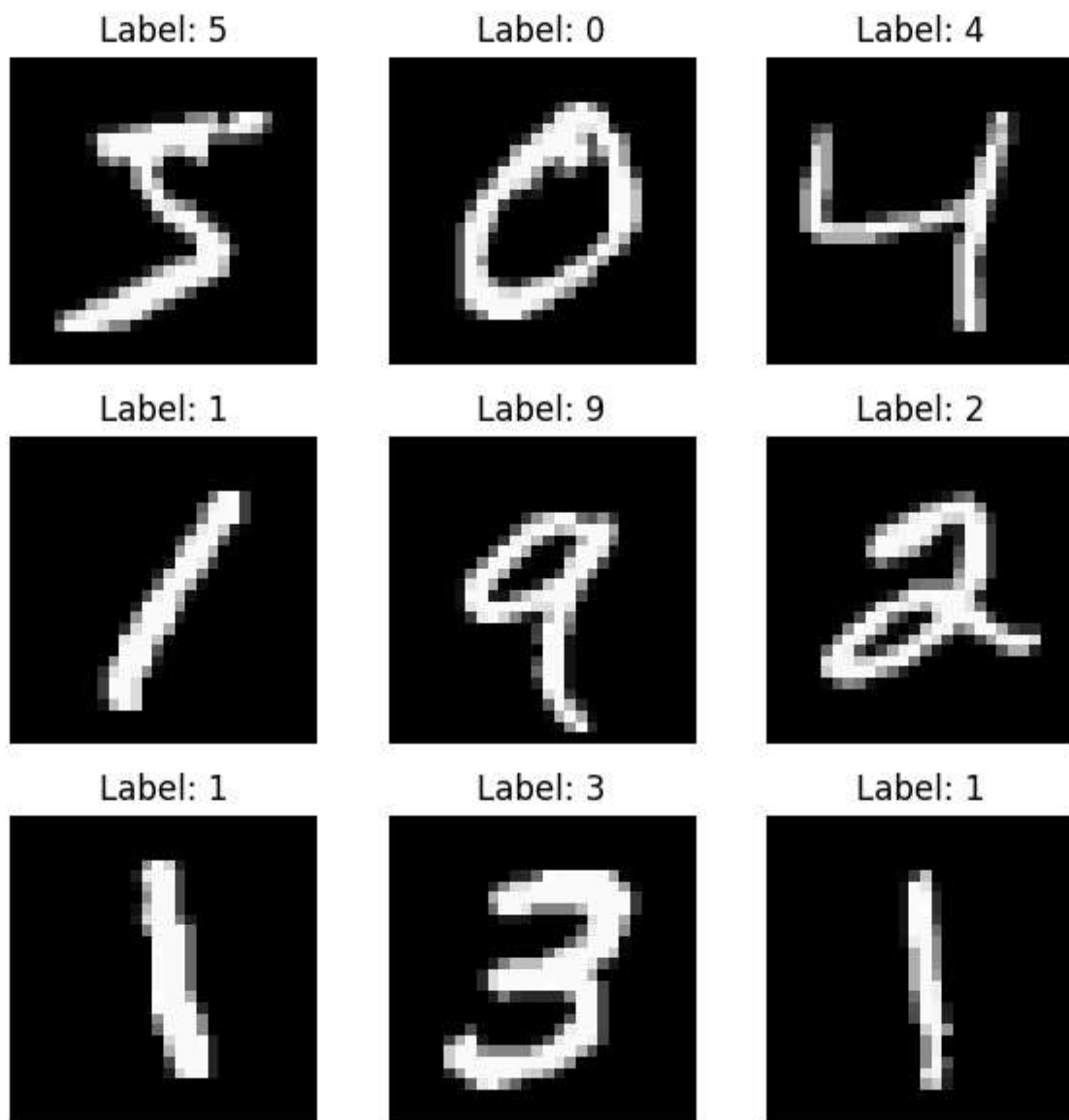
```
x_train = x_train.astype("float32") / 255
x_test = x_test.astype("float32") / 255
```

```
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
```

```
print("x_train shape:", x_train.shape)
print("y_train shape:", y_train.shape)
```

x_train shape: (60000, 28, 28, 1)
y_train shape: (60000, 10)

```
plt.figure(figsize=(6, 6))
for i in range(9):
    plt.subplot(3, 3, i + 1)
    plt.imshow(x_train[i].reshape(28, 28), cmap='gray')
    plt.title(f"Label: {np.argmax(y_train[i])}")
    plt.axis('off')
plt.tight_layout()
plt.show()
```



```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

model = Sequential([
    Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape=(28, 28, 1)),
    MaxPooling2D(pool_size=(2, 2)),

    Conv2D(64, kernel_size=(3, 3), activation='relu'),
    MaxPooling2D(pool_size=(2, 2)),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(10, activation='softmax')
])
model

```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
<Sequential name=sequential_1, built=True>
```

```
model.compile(optimizer='adam',loss='categorical_crossentropy',metrics=['accu
```

```
history = model.fit(
    x_train, y_train,
    epochs=5,
    batch_size=128,
    validation_split=0.1,
    verbose=2
)
```

```
Epoch 1/5
422/422 - 43s - 101ms/step - accuracy: 0.8998 - loss: 0.3300 - val_accuracy: 0.9
Epoch 2/5
422/422 - 42s - 100ms/step - accuracy: 0.9693 - loss: 0.1034 - val_accuracy: 0.9
Epoch 3/5
422/422 - 87s - 206ms/step - accuracy: 0.9778 - loss: 0.0754 - val_accuracy: 0.9
Epoch 4/5
422/422 - 52s - 123ms/step - accuracy: 0.9823 - loss: 0.0604 - val_accuracy: 0.9
Epoch 5/5
422/422 - 55s - 129ms/step - accuracy: 0.9840 - loss: 0.0534 - val_accuracy: 0.9
```

```
test_loss, test_acc = model.evaluate(x_test, y_test)
print(f"Test Accuracy: {test_acc:.4f}")
print(f"Test Loss: {test_loss:.4f}")
```

```
313/313 ————— 3s 10ms/step - accuracy: 0.9891 - loss: 0.0299
Test Accuracy: 0.9891
Test Loss: 0.0299
```

```
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
```

```
y_pred = model.predict(x_test)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true = np.argmax(y_test, axis=1)
```

```
print("Classification Report:\n")
print(classification_report(y_true, y_pred_classes))
```

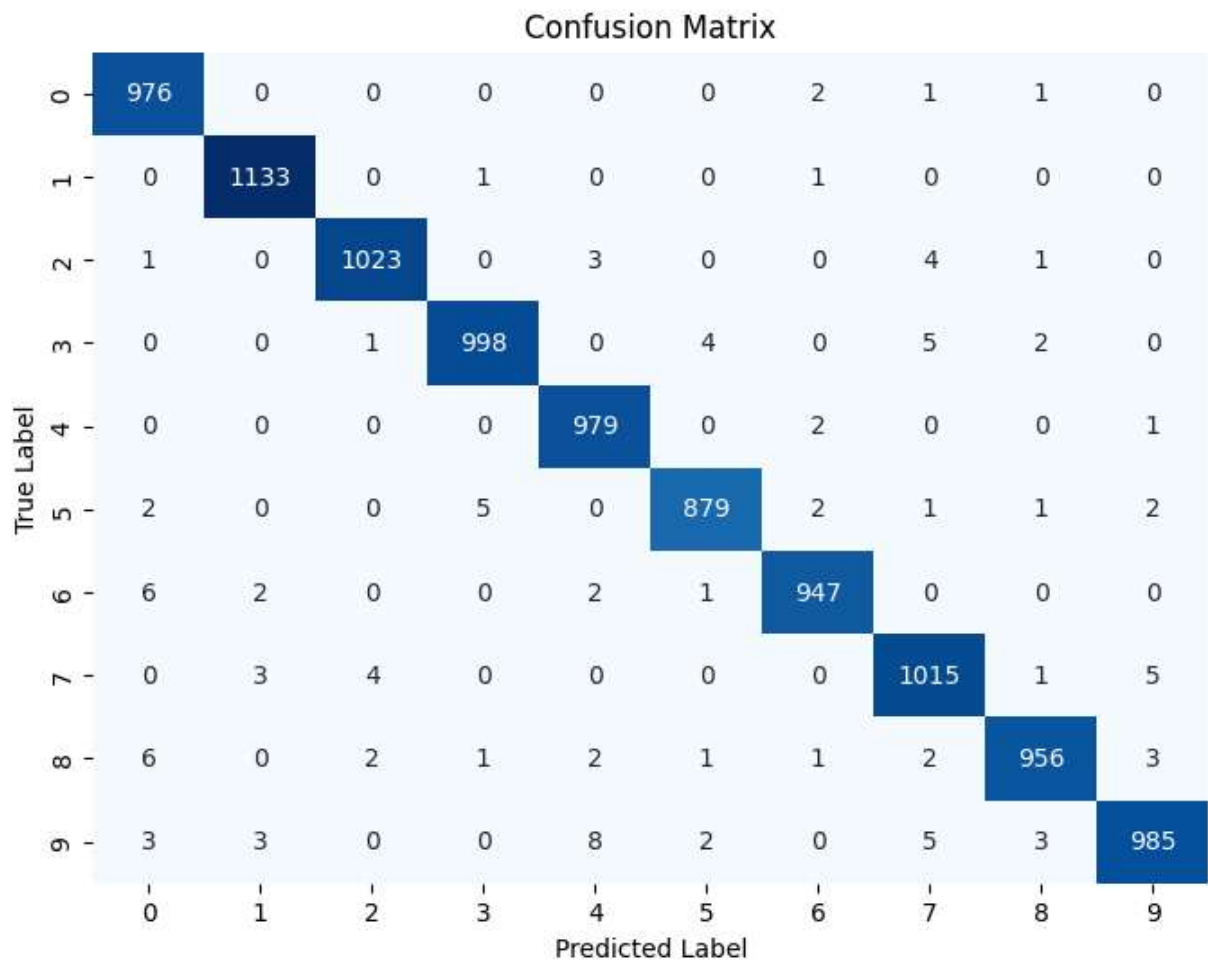
```
313/313 ————— 3s 11ms/step
Classification Report:
```

```
precision    recall  f1-score   support
```

0	0.98	1.00	0.99	980
1	0.99	1.00	1.00	1135
2	0.99	0.99	0.99	1032
3	0.99	0.99	0.99	1010
4	0.98	1.00	0.99	982
5	0.99	0.99	0.99	892
6	0.99	0.99	0.99	958
7	0.98	0.99	0.98	1028
8	0.99	0.98	0.99	974
9	0.99	0.98	0.98	1009
accuracy			0.99	10000
macro avg	0.99	0.99	0.99	10000
weighted avg	0.99	0.99	0.99	10000

```
conf_matrix = confusion_matrix(y_true, y_pred_classes)
```

```
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', cbar=False)
plt.title("Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()
```



```

predictions = model.predict(x_test)

predicted_classes = np.argmax(predictions, axis=1)
true_classes = np.argmax(y_test, axis=1)

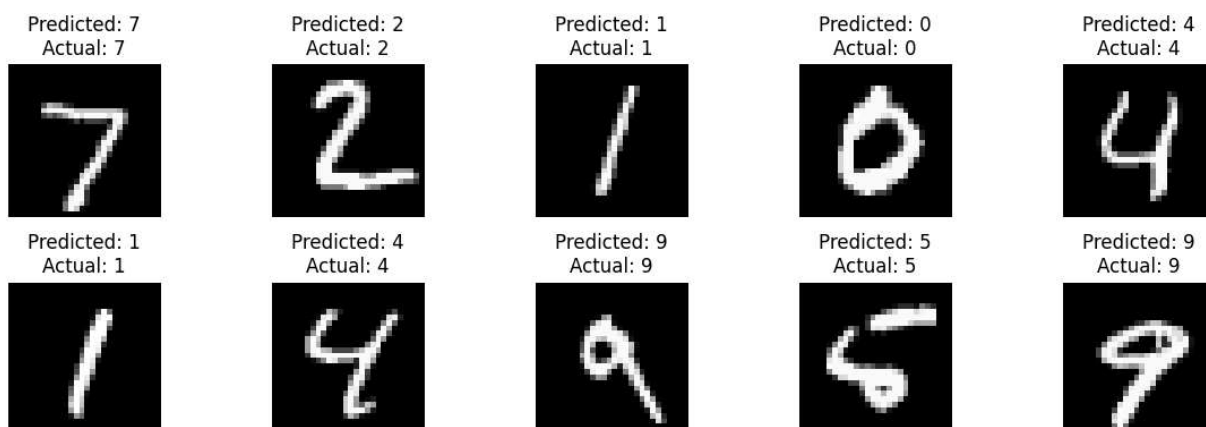
```

313/313 ————— 5s 15ms/step

```

plt.figure(figsize=(12, 4))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
    plt.title(f"Predicted: {predicted_classes[i]}\nActual: {true_classes[i]}")
    plt.axis('off')
plt.tight_layout()
plt.show()

```



```

index = 500
single_image = x_test[index].reshape(1, 28, 28, 1)

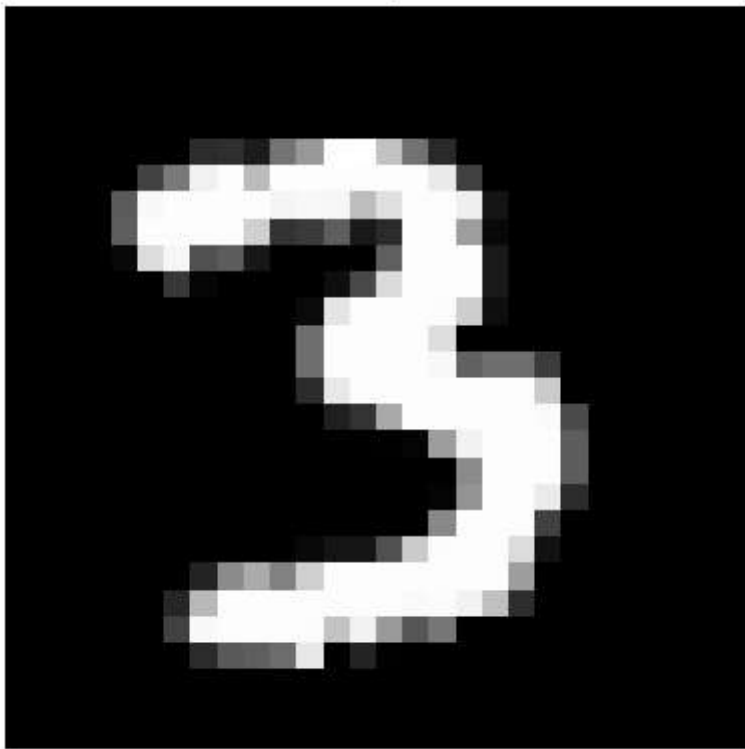
predicted_label = np.argmax(model.predict(single_image))
actual_label = np.argmax(y_test[index])

plt.imshow(x_test[index].reshape(28, 28), cmap='gray')
plt.title(f"Predicted: {predicted_label}, Actual: {actual_label}")
plt.axis('off')
plt.show()

```

1/1 — 0s 38ms/step

Predicted: 3, Actual: 3



Start coding or [generate](#) with AI.